

1. Wired LAN, Wireless LAN, and VLAN

(a) Wired LAN

- **Definition:** A Local Area Network (LAN) that uses physical cables (twisted pair, coaxial, fiber optic) to connect devices like PCs, servers, and printers.
 - **Standards:** Ethernet (IEEE 802.3).
 - **Features:**
 - High speed (10 Mbps → 100 Gbps).
 - Secure, as physical access is required.
 - Reliable and low latency.
 - **Limitation:** Not flexible; cabling cost is high.
 - **Use Case:** Offices, banks, data centers.
-

(b) Wireless LAN (WLAN)

- **Definition:** A LAN that connects devices through wireless signals (Wi-Fi).
 - **Standards:** IEEE 802.11 family.
 - **Features:**
 - Mobility → users can connect anywhere within coverage.
 - Easy installation, no cabling needed.
 - Uses **CSMA/CA** to avoid collisions.
 - **Limitation:** Less secure than wired; prone to interference.
 - **Use Case:** Universities, airports, homes, cafes.
-

(c) Virtual LAN (VLAN)

- **Definition:** A logical subgroup of a LAN, created within a switch, to separate broadcast domains without requiring separate physical networks. (IEEE 802.1Q standard).
- **How it Works:** Devices are grouped by function, team, or application → not by location.
- **Features:**
 - Better security (isolates sensitive departments).

- Reduces congestion (smaller broadcast domains).
 - Flexible network management.
 - **Example:** In a university LAN, Finance dept. and Students' lab are separated into VLANs even though they share the same switch.
-

 Exam Note:

- **Wired LAN → secure & high speed.**
- **Wireless LAN → mobility & flexibility.**
- **VLAN → logical separation for efficiency and security.**

1. Data Link Layer and its Functions

The **Data Link Layer (DLL)** is the **second layer of the OSI model**. It ensures **reliable and efficient data transfer** between two nodes connected on the same local network. It provides error detection, flow control, and access control to make communication error-free.

Key Functions:

1. **Framing** – Divides data from the network layer into frames and adds headers/trailers with MAC addresses and error-checking codes.
2. **Physical Addressing (MAC Addressing)** – Uses **MAC addresses** to deliver frames to the correct device.
3. **Flow Control** – Prevents the sender from overwhelming the receiver by regulating data transmission speed.
4. **Error Control** – Detects and corrects errors (e.g., using CRC) to ensure data integrity.
5. **Medium Access Control (MAC)** – Manages how multiple devices share the communication channel and avoids collisions.

2. Multiplexing in Computer Networks

Multiplexing allows **multiple signals to share a single communication channel**, improving efficiency. The main types are:

1. Frequency Division Multiplexing (FDM)

- **How it works:** Channel bandwidth is divided into non-overlapping frequency bands. Each signal uses a different carrier frequency.
- **Analogy:** Like cars in different lanes on a highway, each lane = frequency band.
- **Use cases:** Radio broadcasting, Cable TV (CATV), analog systems.

2. Time Division Multiplexing (TDM)

- **How it works:** The channel is divided into fixed **time slots**, and each signal transmits in its slot.
 - **Types:**
 - **Synchronous TDM:** Fixed slots (may waste bandwidth).
 - **Asynchronous/Statistical TDM:** Slots assigned dynamically (more efficient).
 - **Use cases:** Digital telephony, multiplexing voice/data streams.
-

3. Wavelength Division Multiplexing (WDM)

- **How it works:** Optical version of FDM; combines multiple light signals of different **wavelengths (colors)** into one optical fiber.
 - **Analogy:** Like multiple colored beams traveling in the same fiber without interfering.
 - **Use cases:** High-capacity, long-distance fiber optic communication.
-

✓ Short Note for Revision:

- **FDM** → Different frequencies.
- **TDM** → Different time slots.
- **WDM** → Different light wavelengths in fiber optics.

3.

1. Spread Spectrum (The Core Idea)

A technique that spreads a signal over a much wider frequency band than required.

- **Goal:** To achieve secure and reliable communication that resists interference and eavesdropping.
 - **How:** It mixes the data with a unique "spreading code" known only to the sender and receiver.
-

2. Frequency Hopping Spread Spectrum (FHSS)

The signal rapidly "hops" between many different frequencies in a pre-agreed, pseudo-random pattern.

- **How it works:** The transmitter and receiver are synchronized to jump between frequencies together.
 - **Key Advantage:** Excellent at avoiding narrowband interference (if one channel is blocked, it just hops to another).
 - **Main Example: Bluetooth.**
-

3. Direct Sequence Spread Spectrum (DSSS)

Each data bit is replaced by a longer bit pattern called a "chipping code," spreading the signal across a single, wide frequency band.

- **How it works:** The data is multiplied by the high-rate chipping code.
- **Key Advantage:** Allows for higher data rates and has good overall resistance to jamming.
- **Main Examples:** Early **Wi-Fi (802.11b)** and **GPS**.

Error control

Error control in the data link layer is a set of techniques used to **detect and correct errors** that can occur when data frames are transmitted over a network channel. The primary goal is to ensure that the data received by the destination device is identical to the data sent by the source device, thus providing reliable data delivery across a physical link. reliable delivery.

This process is generally accomplished in two main steps: **error detection** and **error correction**.

Error Detection vs Error Correction (in DLL)

Aspect	Error Detection	Error Correction
Meaning	Identifies whether errors have occurred in transmitted frames.	Identifies and fixes the errors in the received frames.
Purpose	To ensure receiver knows data is corrupted.	To deliver the correct data without needing retransmission (if possible).
Techniques	Parity bits, Checksum, CRC.	Forward Error Correction (FEC), Automatic Repeat Request (ARQ).
Action Taken	Receiver requests retransmission when error is detected.	Receiver corrects errors directly (or requests retransmission if uncorrectable).
Overhead	Adds fewer redundant bits.	Adds more redundancy for correction capability.
Example	CRC detects bit errors in Ethernet.	Hamming Code corrects single-bit errors.

Error Detection

Error detection involves adding redundant data to a frame, which allows the receiver to check if any bits were flipped during transmission. If an error is detected, the receiver typically discards the frame. The most common methods are:

1. Parity Check

This is the simplest method. A single **parity bit** is added to a block of data.

- **Even Parity:** The parity bit is set to 1 if the number of ones in the data is odd, making the total number of ones (including the parity bit) even.
- **Odd Parity:** The parity bit is set to 1 if the number of ones in the data is even, making the total number of ones odd.

Limitation: It can only detect an odd number of bit errors (1, 3, 5, etc.). It cannot detect an even number of bit errors.

2. Checksum

The sender divides the data into fixed-size segments. It then adds all the segments together using 1's complement arithmetic and appends the resulting sum (the checksum) to the data. The receiver performs the same calculation on the received data and compares its result with the received checksum. If they match, the data is considered error-free.

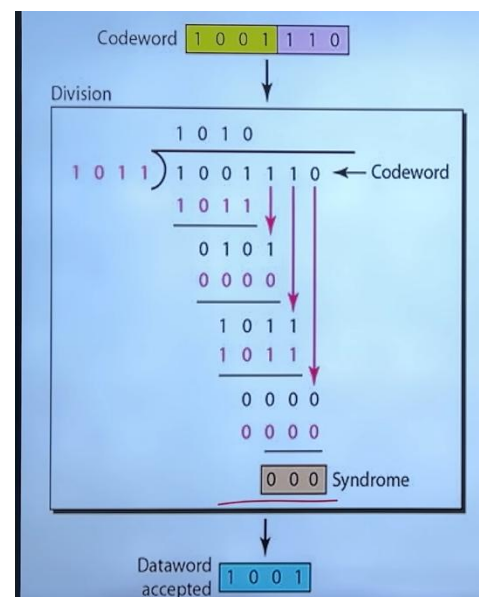
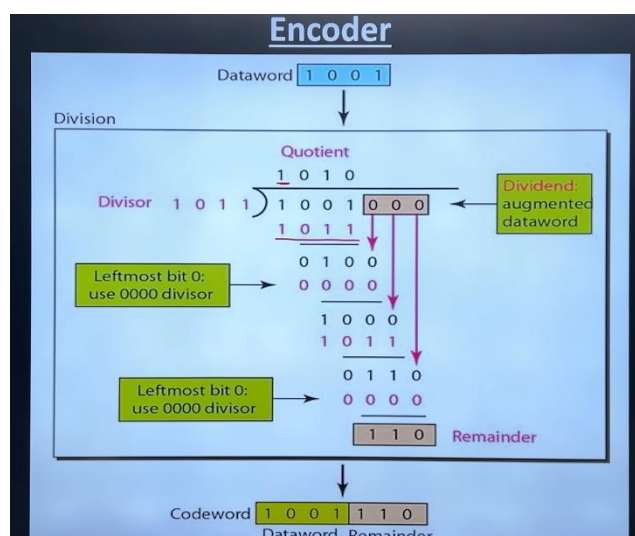
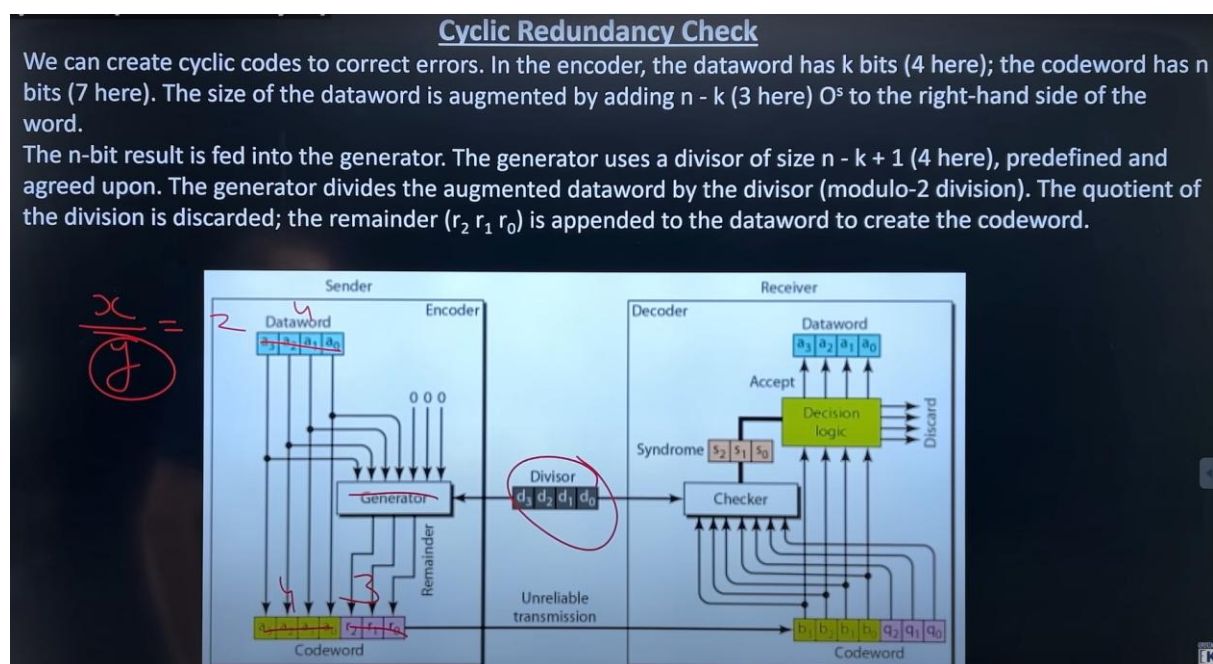
Use Case: Often used in higher-level protocols like TCP/IP.

3. Cyclic Redundancy Check (CRC)

CRC is a more powerful and common error detection technique. The sender and receiver agree on a predetermined binary number called a **Generator Polynomial**.

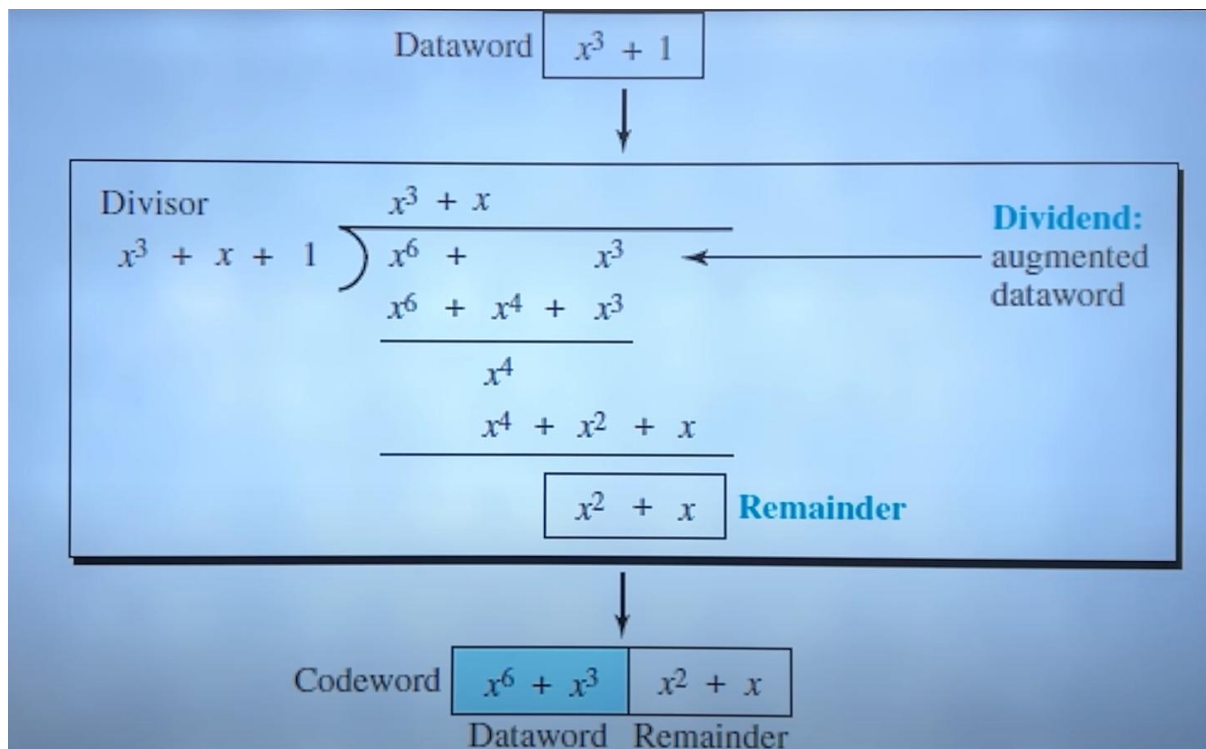
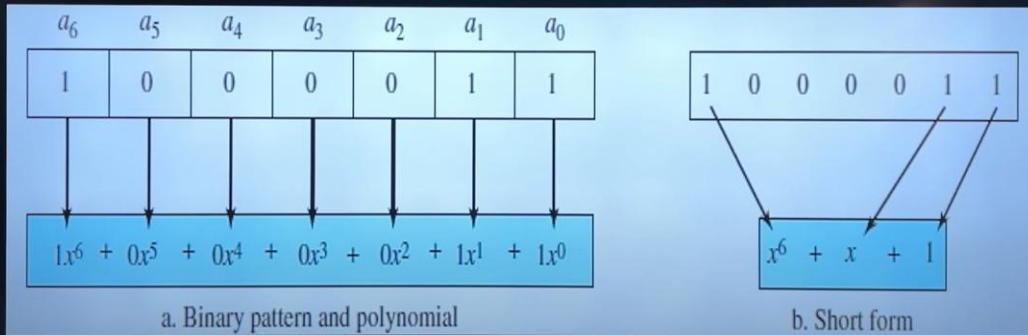
The sender performs a binary division of the data by the generator polynomial and appends the remainder to the end of the data. The receiver divides the entire received frame (data + remainder) by the same generator polynomial. If the remainder of this division is zero, the data is accepted; otherwise, it is rejected as corrupted.

Advantage: CRC is highly effective at detecting single-bit errors, burst errors (multiple consecutive bit errors), and most other common error patterns.



Polynomials

A pattern of 0's and 1's can be represented as a **polynomial** with coefficients of 0 and 1. The power of each term shows the position of the bit; the coefficient shows the value of the bit. An advantage is that a large binary pattern can be represented by short terms.



Error Correction

Once an error is detected, the frame needs to be corrected. This is typically handled by having the receiver request the sender to retransmit the corrupted or lost frame. This process is known as **Automatic Repeat Request (ARQ)**.

1. Stop-and-Wait ARQ

This is the simplest ARQ method.

- The sender sends one frame and then waits for an **acknowledgment (ACK)** from the receiver.

- If an ACK is received, the sender sends the next frame.
- If the ACK is not received within a certain time (timeout) or a **negative acknowledgment (NAK)** is received, the sender retransmits the same frame.

Drawback: It's inefficient because the sender is idle while waiting for the acknowledgment.

2. Go-Back-N ARQ

This method improves efficiency by allowing the sender to transmit multiple frames (up to a window size of 'N') without waiting for an ACK for each one.

- The receiver only accepts frames in the correct order.
- If a frame is lost or corrupted, the receiver discards that frame and all subsequent frames.
- The sender, upon timeout or receiving a NAK, **retransmits the corrupted frame and all the frames that came after it.**

3. Selective Repeat ARQ

This is the most efficient ARQ protocol. The sender can also transmit multiple frames up to a certain window size.

- The receiver can accept and buffer out-of-order frames.
- If a frame is detected as an error, the receiver sends a NAK for **only that specific frame.**
- The sender retransmits **only the single frame that was lost or corrupted**, not all the subsequent frames. This saves a significant amount of bandwidth compared to Go-Back-N.

Flow Control

Flow control is about managing the **rate of data transmission** between a sender and a receiver. Its main purpose is to prevent a fast sender from overwhelming a slow receiver with too much data at once.

Flow Control Protocols

The primary goal of flow control is to pace the sender. The two main methods are:

1. Stop-and-Wait

This is the simplest form of flow control.

- The sender transmits a single frame and then **stops and waits** for an acknowledgment (ACK) from the receiver.
- The sender cannot send the next frame until the ACK for the current one is received.

Analogy: Having a conversation where you say one sentence and wait for a "got it" before saying the next. It's simple but very slow, especially over long-distance links.

2. Sliding Window

This is a more efficient method that allows the sender to transmit multiple frames without waiting for an individual ACK for each one.

- Both sender and receiver maintain a "window" of a fixed size, which represents the number of frames that can be sent.
- The sender can transmit all the frames in its window. As the receiver sends ACKs, the window "slides" forward, allowing the sender to transmit new frames.

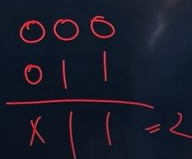
Analogy: A conveyor belt between the sender and receiver. The sender can place several items (frames) on the belt. As the receiver picks them up and gives a thumbs-up (ACK), the sender is allowed to place more items on the belt. The size of the belt is the window size.

Hamming Distance

The **Hamming distance** is a simple metric that measures the difference between two strings of **equal length**. It's the number of positions at which the corresponding characters (or bits) are different.

Hamming Distance

- Hamming distance, represented as $d(x, y)$, is a measure of the difference between two words of equal size, calculated by counting the number of dissimilar bits at the corresponding positions in the two words.
- To find the Hamming distance, perform an XOR operation on the two words and count the number of 1s in the outcome.
- For instance, the Hamming distance $d(000, 011)$ equals 2, illustrated through the XOR operation: $000 \text{ XOR } 011$ equals 011 , which has two 1s.
- The Hamming distance $d(10101, 11110)$ is _____ ?



The image shows a handwritten calculation for the Hamming distance between 000 and 011. It is written as follows:

$$\begin{array}{r} 000 \\ 011 \\ \hline 011 \end{array}$$

Below this, it says $X | 1 = 2$, indicating that the result 011 has two 1s, which is the Hamming distance.

Random Access Protocols

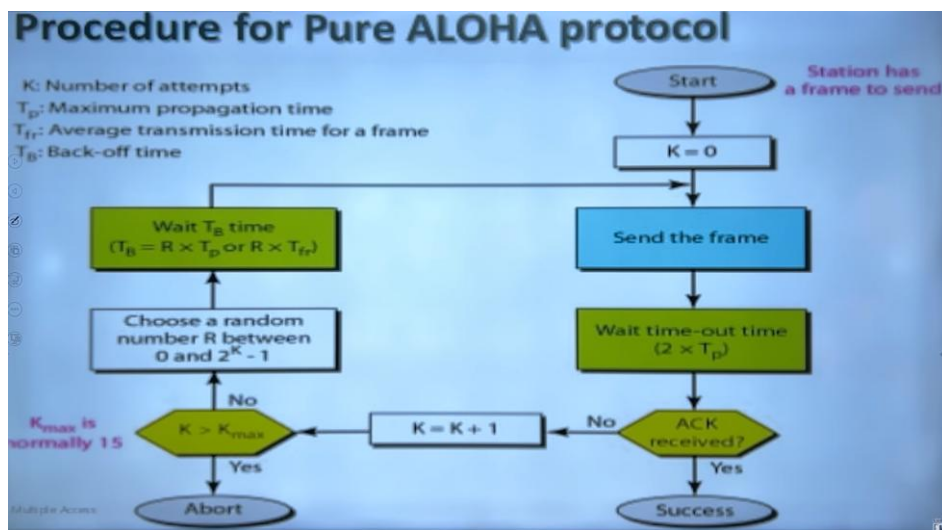
Random Access Protocols (also known as contention-based protocols) are a set of rules where each device has the freedom to send data whenever it has something to send. There's no fixed schedule or master controller telling devices when it's their turn.

The core idea is "send when you're ready, but be prepared for collisions." A **collision** happens when two or more devices transmit simultaneously, causing their signals to interfere and become unreadable. These protocols are all about how to minimize the chance of collisions and how to recover when they inevitably happen.

1. ALOHA

This is the simplest random access protocol.

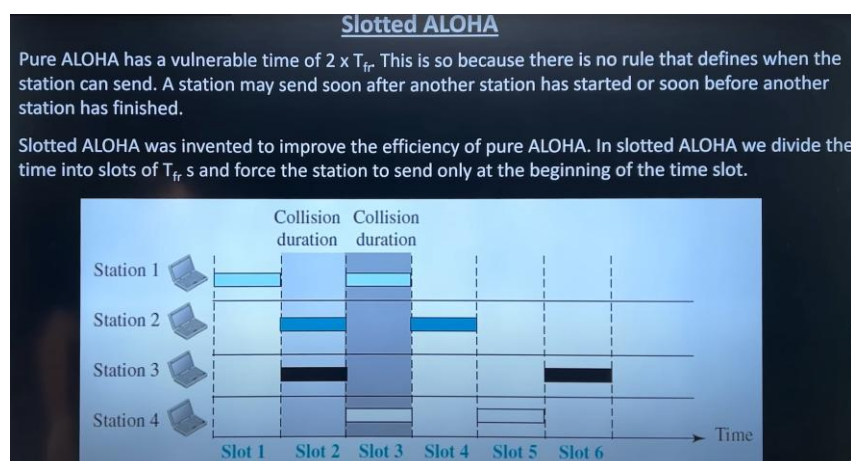
- **The Rule:** "If you have data, send it."
- **Collision Handling:** After sending, the sender waits for an acknowledgment. If no acknowledgment is received within a certain time, it assumes a collision occurred and waits a random amount of time before retransmitting.
- **Problem:** It's chaotic. On a busy channel, many collisions occur, and efficiency is very low (around 18%).



2. Slotted ALOHA

A slight improvement over ALOHA.

- **The Rule:** The channel is divided into fixed-size time slots. A device can only begin transmitting at the *start* of a time slot.
- **Collision Handling:** Same as ALOHA, but because transmissions are synchronized to slots, collisions are less likely.
- **Improvement:** This simple change doubles the efficiency to about 37%.



3. Carrier Sense Multiple Access (CSMA)

A significant improvement that adds a "listen before you talk" rule.

- **The Rule:** Before transmitting, a device "senses" the channel to check if it's idle. If the channel is busy, the device waits.
- **Problem:** Collisions can still happen due to **propagation delay**. Two devices at opposite ends of a long cable might both sense the channel as idle, start sending, and then have their frames collide in the middle.

4. CSMA/CD (Collision Detection)

The "listen while you talk" protocol, famously used in classic **Ethernet**.

- **The Rule:** It follows CSMA (listen before talk), but devices also listen *while* they are transmitting.
- **Collision Handling:** If a device detects that its signal is interfering with another signal (a collision), it immediately stops transmitting, sends a "jam signal" to ensure all other devices know a collision occurred, and then waits a random amount of time before trying again (this is called **binary exponential backoff**).

5. CSMA/CA (Collision Avoidance)

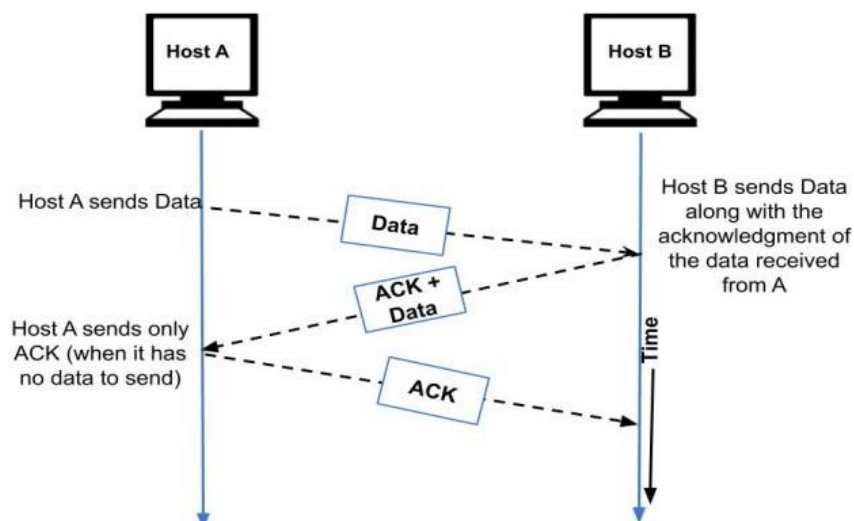
The "try to avoid collisions" protocol, famously used in **Wi-Fi (802.11)**.

- **The Rule:** It uses CSMA (listen before talk). Since wireless devices can't easily detect collisions while transmitting (due to the "hidden station problem"), they focus on *avoiding* them.
- **Collision Avoidance:** Before sending, a device can send a short **Request to Send (RTS)** signal. The access point then replies with a **Clear to Send (CTS)** signal that all nearby devices can hear. This tells other devices to wait, clearing the channel for the original sender. This helps prevent collisions before they happen.

Piggybacking

Piggybacking is a technique used in two-way communication to improve network efficiency by sending an acknowledgment (ACK) along with an outgoing data frame.

Instead of sending a separate acknowledgment frame, the acknowledgment is "pigged back" or attached to the next data frame being sent in the reverse direction.



Ethernet

Ethernet is the most widely used technology for building wired **Local Area Networks (LANs)**. It defines the rules for how devices on a network format and transmit data so other devices on the same network can recognize, receive, and process it.

Key Concepts

- **MAC Address:** Every Ethernet network interface card (NIC) has a unique 48-bit Media Access Control (MAC) address burned into it. This is used to identify the source and destination of data frames on the local network.
- **Ethernet Frame:** Data is sent in packets called **frames**. Each frame contains the source and destination MAC addresses, the data payload, and error-checking information (like CRC).

Example. given a received code word with one week error, apply hammming code to detect and correct code

♦ What is Hamming Code?

Hamming Code is a method that helps us:

1. **Detect** if there is an error in received data.
2. **Correct** the error if only **one bit** is wrong.

It works by adding **extra bits (parity bits)** to the original data. These parity bits help us find where the error is.

♦ Step 1: Positions of bits

Suppose we send **4 data bits** → d1 d2 d3 d4

We add **3 parity bits** → p1 p2 p4

So total **7 bits** → positions 1 to 7.

- Parity bits go to positions: 1, 2, 4 (powers of 2).
- Data bits go to positions: 3, 5, 6, 7.

So order is:

[p1 p2 d1 p4 d2 d3 d4]

Step 2: Example with 4-bit data

Say our data is 1011.

We place it with parity bits:

Position 1 2 3 4 5 6 7

Bit p1 p2 1 p4 0 1 1

So positions 1, 2, 4 are parity bits (p1, p2, p4).

Step 3: Calculate Parity Bits

We use **even parity** (count of 1s must be even).

- **p1 checks positions 1,3,5,7** → (p1,1,0,1) → currently (1+0+1 = 2 ones). Even → p1 = 0
- **p2 checks positions 2,3,6,7** → (p2,1,1,1) → (1+1+1 = 3 ones). Odd → p2 = 1 (to make even)
- **p4 checks positions 4,5,6,7** → (p4,0,1,1) → (0+1+1 = 2 ones). Even → p4 = 0

So final transmitted code = **0110011**

Step 4: Suppose an Error Happens

Say bit 5 flips (0 → 1). Received code = 0110111.

Now we recalc parity at receiver:

- **Check p1 (1,3,5,7 → 0,1,1,1)** → 3 ones → error → p1 = 1
- **Check p2 (2,3,6,7 → 1,1,1,1)** → 4 ones → OK → p2 = 0
- **Check p4 (4,5,6,7 → 0,1,1,1)** → 3 ones → error → p4 = 1

Error position = (p4p2p1) = (101) = 5 

So bit 5 is wrong → Flip it back → Corrected code = 0110011

Example. suppose a message is transmitted with even parity what happens if 2 bits flipped

When **even parity** is used, the parity bit is chosen so that the total number of 1s in the transmitted codeword is even.

- **Single-bit error:** Always detected, because it changes the parity from even to odd.
- **Two-bit error:** May **not be detected**, because flipping two bits still keeps the total number of 1s even, and the receiver assumes the frame is correct.

Conclusion:

- Even parity detects all **odd number of errors** (1, 3, 5 ...).
- Even parity **fails** to detect errors when an **even number of bits** are corrupted (e.g., 2-bit errors).

Example. apply CRC to detect an error by taking any data word and divisor

Given:

- Dataword = 1101
- Divisor (generator) = 1011 (degree = 3)

1. Append zeros (degree = 3)

1101 → 1101 000 (work with 1101000)

2. Do modulo-2 division (binary XOR, no carries)

Perform binary division of 1101000 by 1011. (You may show long-division steps in answer sheet.) The remainder (CRC) computed = **001**.

3. Transmitted codeword = dataword + remainder =

1101 + 001 = **1101001**

(Quick division steps you can write in the exam)

- $1101 \oplus 1011 = 0110 \rightarrow$ bring down next bit $\rightarrow 01100$
- continue XORing as in long division $\rightarrow$ final remainder = 001

4. At receiver (no error):

Receiver divides received 1101001 by 1011 $\rightarrow$ remainder = 000 $\rightarrow$ **no error**.

5. If one bit is flipped in transmission, e.g. received = 1101011 (bit changed)

Receiver divides 1101011 by 1011 $\rightarrow$ remainder = 010 ($\neq$ 000) $\rightarrow$ **error detected**.

Example. apply stop and wait arq to explain what happen if ack lost during transmission

Stop-and-Wait ARQ (ACK lost case)

- In **Stop-and-Wait ARQ**, the sender transmits **one frame** and waits for an **ACK** before sending the next frame.
- If the **ACK is lost** in transmission:
 1. The **receiver** has already received the frame correctly and delivered it to the network layer.
 2. But since the sender does **not receive the ACK** within the timeout period, it assumes the frame (or ACK) was lost.
 3. The **sender retransmits** the same frame.
 4. The **receiver identifies duplicate frames** using the **sequence number (0/1)** and discards the duplicate, but sends ACK again.
 5. Transmission continues normally.

Unit 3. Network Layer

The **Network Layer** (Layer 3 in the OSI model) is the heart of any large network like the internet. Its primary job is to move data **packets** from a source host on one network to a destination host on a *different* network.

Key Responsibilities of the Network Layer

1. Logical Addressing

The Network Layer assigns a unique logical address to every host on the network. This address must be valid across the entire internetwork, not just the local segment.

- **The Protocol:** The **Internet Protocol (IP)** is the most common standard. This is where **IP addresses** (like 192.168.1.10 for IPv4 or 2001:0db8::8a2e:0370:7334 for IPv6) come from.

2. Routing

This is the process of determining the best path for a packet to travel from source to destination. Routers build and maintain **routing tables** (like a map) to make these decisions.

- **How it works:** Routing protocols (like OSPF or BGP) help routers share information and discover the optimal paths through the complex web of interconnected networks.

3. Forwarding

While routing is the process of creating the map, forwarding is the router's action of actually moving a packet. A router receives a packet on an input port, looks at the destination IP address, consults its routing table, and forwards the packet to the correct output port to send it on its next "hop."

4. Internetworking

This is the fundamental purpose of the Network Layer: to connect different, independent networks (which could be using different Layer 2 technologies like Ethernet and Wi-Fi) and make them function as a single, seamless internetwork.

Switching Techniques

Switching techniques are the methods used by networking devices to transmit data across a network. The primary goal of any switching technique is to efficiently move information from a source to a destination by establishing a temporary path between them.

In a large network, it's not practical to have a dedicated physical connection between every pair of devices. Instead, devices connect to "switches," which create temporary connections to route the data. The way these connections are established and how data is handled defines the switching technique.

The three main switching techniques are:

Of course. Here is a shorter explanation of the main switching techniques.

1. Circuit Switching 📞

A **dedicated path** is established between the sender and receiver *before* any data is sent. This path is reserved for the entire communication session.

- **Analogy:** A traditional telephone call. The line is exclusively yours until you hang up.
 - **Key Feature:** Guaranteed bandwidth but can be inefficient if no data is being sent.
-

2. Packet Switching 📧

Data is broken down into small **packets**. Each packet is sent independently, possibly taking a different route to the destination, where they are reassembled.

- **Analogy:** Sending a book chapter by chapter through the mail. Each envelope (packet) finds its own way.
 - **Key Feature:** Very efficient use of the network. This is how the **internet** works.
-

3. Message Switching

The **entire message** is sent from one node to the next, being stored and then forwarded at each intermediate stop.

- **Analogy:** Sending an email or a telegraph. The whole message is received at one point before being sent to the next.
- **Key Feature:** High delay (latency); mostly a predecessor to packet switching.

Internet Protocol

IP stands for **Internet Protocol**. It's the set of rules that governs how data is sent from one computer to another across the internet.

The main job of IP is to provide a unique **logical address** to every device on a network, much like a mailing address for a house. This **IP address** ensures that data packets are delivered to the correct destination.

Key Functions of IP

1. Addressing

IP assigns a unique address to each device. This allows routers to identify the source and destination of data packets. There are two main versions in use:

- **IPv4:** The older, more common version, which uses a 32-bit address (e.g., 192.168.1.1). It supports about 4.3 billion unique addresses.
- **IPv6:** The newer version, designed to solve the shortage of IPv4 addresses. It uses a 128-bit address (e.g., 2001:0db8:85a3:0000:0000:8a2e:0370:7334), providing a virtually limitless number of addresses.

2. Routing

IP is responsible for moving data packets from their source to their destination across multiple networks. Routers examine the destination IP address in each packet's header to decide where to forward it next on its journey.

Analogy: 📮 Think of the postal service. You write a full address (the IP address) on an envelope (the data packet). Each post office (a router) looks at the destination address to figure out which city to send it to next, until it finally reaches the correct mailbox.

IP is a **connectionless** and "**best-effort**" protocol. This means it doesn't guarantee that packets will arrive, or that they will arrive in the correct order. That responsibility is left to higher-level protocols like TCP.

Logical addressing

is a method of assigning a unique address to a device on a network that is independent of the physical hardware. This address allows a device to be identified across different interconnected networks.

The primary purpose of a logical address is to enable **routing**. It provides a global address that routers can use to find the best path to deliver data packets from a source network to a destination network.

Logical vs. Physical Addressing

It's crucial to distinguish between a logical address and a physical address:

- **Logical Address (e.g., IP Address):** This is a software-based address assigned at the Network Layer (Layer 3). It can change depending on which network the device is connected to. It's like your home's mailing address, which tells the postal service which city and neighborhood to deliver a letter to.
 - **Example:** 192.168.1.101 (IPv4)
- **Physical Address (MAC Address):** This is a hardware-based address burned into the Network Interface Card (NIC) at the Data Link Layer (Layer 2). It is permanent and only used for communication within the same local network segment. It's like the name of a person within a household.
 - **Example:** 00:1A:2B:3C:4D:5E

Analogy: 📧 Think of sending a letter to "John Smith at 123 Main St, Anytown."

- The **logical address** is "123 Main St, Anytown". The postal system (routers) uses this to get the letter to the right house.
- The **physical address** is "John Smith". Once the mail carrier arrives at the house, this name is used to identify the specific recipient.

IPv4 and IPv6

are two versions of the Internet Protocol, which provides the unique addresses devices use to communicate over the internet. The main difference between them is that **IPv6 uses a larger address space** to solve the problem of address exhaustion that plagues IPv4.

Address Size and Format

- **IPv4:** Uses a **32-bit** address, allowing for approximately 4.3 billion unique addresses.
 - **Format:** Written as four decimal numbers separated by dots (e.g., 192.168.1.1).
- **IPv6:** Uses a **128-bit** address, providing a virtually limitless number of addresses (340 undecillion).
 - **Format:** Written as eight groups of four hexadecimal digits separated by colons (e.g., 2001:0db8:85a3:0000:0000:8a2e:0370:7334).

Feature	IPv4	IPv6
Address size	32 bits (4 bytes) → about 4.3 billion addresses	128 bits (16 bytes) → virtually unlimited addresses
Address format	Dotted decimal (e.g., 192.168.0.1)	Hexadecimal, colon-separated (e.g., 2001:0db8::1)
Header size	20–60 bytes (variable)	Fixed 40 bytes (simpler)
Security	Optional (IPSec not mandatory)	Built-in support for IPSec
Configuration	Manual or DHCP	Auto-configuration (stateless) + DHCPv6
Broadcast	Supports broadcast	No broadcast (uses multicast & anycast)
Fragmentation	Done by sender and routers	Done only by sender
Quality of Service (QoS)	Uses Type of Service (ToS) field	Uses Flow Label field (better QoS support)
Deployment	Widely used, but addresses nearly exhausted	Newer, designed to replace IPv4

What is Subnetting?

- **Subnetting** is the process of **dividing a large network into smaller, manageable networks** called **subnets**.
- Each subnet has its own **Network ID** and **Host IDs**.
- Helps in **efficient use of IP addresses** and better **network management**.

◆ Why Subnetting?

1. **Saves IP addresses** → no need to waste addresses in large blocks.
2. **Improves network performance** → reduces traffic in each subnet.
3. **Easier management** → organize devices by departments, floors, or locations.

◆ **Example:**

Original Network: 192.168.1.0/24 → 256 IP addresses

We want to **divide it into 2 subnets**.

Step 1: Determine new prefix

- Original prefix = /24 → 8 bits for host
 - Dividing into 2 subnets → need **1 extra bit** for subnet
 - New prefix = /25 → 25 bits for network, 7 bits for hosts
-

Step 2: Find subnets

- **Subnet 1:** 192.168.1.0/25 → Host range: 192.168.1.1 to 192.168.1.126
- **Subnet 2:** 192.168.1.128/25 → Host range: 192.168.1.129 to 192.168.1.254

Note: Each subnet has 128 addresses → usable hosts = 126 (network & broadcast not usable)

Subnet Mask

Subnet Masking

- An **IP address** has two parts:
 1. **Network ID** → tells which network
 2. **Host ID** → tells which device inside that network
 - A **Subnet Mask** is used to **separate the Network ID and Host ID** in the IP address.
-

◆ **How it works:**

- Subnet mask = series of 1s (network bits) + series of 0s (host bits).
- Example:

IP Address → 192.168.1.10

Subnet Mask → 255.255.255.0

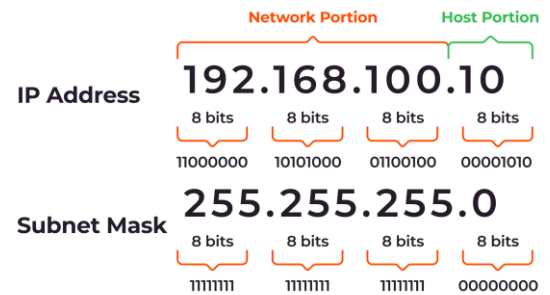
👉 Convert to binary:

- IP = 11000000.10101000.00000001.00001010
- Mask = 11111111.11111111.11111111.00000000

Result:

- First 24 bits (1s) → Network ID = 192.168.1
- Last 8 bits (0s) → Host ID = 10

Binary Notation of IP Address and Subnet



◆ CIDR Notation

- Instead of writing full subnet mask, we write /n where n = number of 1s.
- Example:
 - 255.255.255.0 → /24
 - 255.255.255.128 → /25

CIDR (Classless Inter-Domain Routing)

Definition:

CIDR is a method for **allocating IP addresses** and **routing** Internet traffic introduced in 1993 to replace the old classful addressing system.

It uses **variable-length subnet masks (VLSM)** to make IP address usage more efficient and reduce wastage.

Why CIDR was introduced?

1. To overcome the **limitations of classful addressing** (Class A, B, C).
2. To **reduce wastage** of IP addresses (e.g., Class B had ~65,000 hosts even if you needed only 200).
3. To make **routing tables smaller** by grouping addresses (route aggregation / supernetting).
4. To extend the lifetime of **IPv4**.

CIDR Notation:

- Written as: **IP_address / prefix_length**
- Example: 192.168.1.0/24

- /24 → means first **24 bits are the network ID** (subnet mask = 255.255.255.0).
 - Remaining bits ($32 - 24 = 8$ bits) → for host IDs ($2^8 - 2 = 254$ hosts).
-

How CIDR Works:

1. CIDR ignores fixed classes (A, B, C).
2. Uses **prefix length** to decide how many bits belong to the **network ID** and how many to the **host ID**.
 - Example:
 - 10.0.0.0/8 → First 8 bits = Network, remaining 24 bits = Hosts.
 - 172.16.0.0/16 → First 16 bits = Network, remaining 16 bits = Hosts.

Address mapping

Address mapping is the process of translating an address from one layer of the network model to another. The most common example is mapping a Network Layer **IP address** to a Data Link Layer **MAC address**. Devices need both addresses to communicate: the IP address to find the destination network and the MAC address to deliver the data frame to the specific device on the local network.

Here are the key protocols used for this process.

ARP (Address Resolution Protocol)

ARP is used to find the **MAC address** of a device when you already know its **IP address**. It's the most common address mapping protocol used in modern networks.

- **Question:** "My IP address is 192.168.1.10, and I want to send a packet to 192.168.1.50. What is the MAC address of the device using 192.168.1.50?"
- **How it works:**
 1. The source computer sends an **ARP Request** broadcast message to every device on the local network, asking "Who has the IP address 192.168.1.50?"
 2. All devices on the network receive this broadcast, but only the device with the IP address 192.168.1.50 will respond.
 3. That device sends an **ARP Reply** directly back to the source computer, saying "I have that IP, and my MAC address is 00:1A:2B:3C:4D:5E."
 4. The source computer then stores this mapping in its ARP cache for future use.

RARP (Reverse Address Resolution Protocol)

RARP is an obsolete protocol that does the opposite of ARP. It was used by a device to discover its **own IP address** when it only knew its **MAC address**.

- **Question:** "My MAC address is 00:1A:2B:3C:4D:5E. Can someone please tell me what my IP address is?"
- **How it worked:** A diskless workstation would broadcast a RARP request. A RARP server on the network, which had a manually configured table of MAC-to-IP mappings, would reply with the assigned IP address.
- **Why it's obsolete:** RARP was very limited. It has been completely replaced by BOOTP and DHCP.

BOOTP (Bootstrap Protocol)

BOOTP was created to improve upon RARP. Like RARP, it allows a diskless client to discover its own IP address. However, it is more advanced because it can also provide other crucial information.

- **Question:** "My MAC address is 00:1A:2B:3C:4D:5E. What is my IP address, and also what is the IP address of the default gateway and the boot file I need to load my OS?"
- **How it works:** A client broadcasts a BOOTP request. A BOOTP server on the network looks up the client's MAC address in a pre-configured table and sends back a reply containing the client's IP address, the gateway's IP address, and the location of a boot image file.
- **Limitation:** BOOTP still relied on a static, manually configured mapping table.

DHCP (Dynamic Host Configuration Protocol)

DHCP is the modern, dynamic protocol used to automatically assign IP addresses and other network configuration information to devices. It is an extension of BOOTP and is the standard used today.

- **Request:** "I'm a new device on this network. Please give me an IP address, a subnet mask, a default gateway IP, and DNS server IPs."
- **How it works:**
 1. **Discover:** A client device sends a broadcast message to "discover" any DHCP servers on the network.

2. **Offer:** One or more DHCP servers may respond with an "offer" of an available IP address and other configuration details.
3. **Request:** The client accepts one of the offers and sends a broadcast "request" to that specific server, formally asking to use the offered IP address.
4. **Acknowledge (ACK):** The DHCP server finalizes the lease and sends an acknowledgment, confirming the assignment.

Key Advantage: DHCP is **dynamic**. It manages a pool of IP addresses and leases them to devices for a certain period, automatically reclaiming and reassigning them as needed. This eliminates the need for manual configuration.

Delivery

- **Definition:** The process of delivering a packet from the **last router** to the **final destination (host)**.
- **Types:**
 - **Direct Delivery:** Source and destination are on the same physical network (e.g., same LAN). No router needed.
 - **Indirect Delivery:** Destination is on another network, so the packet goes via one or more routers.

Forwarding

- **Definition:** The process of **sending a packet from one router to the next hop** on the path toward the destination.
- **Key Point:** Forwarding uses the **routing table** to decide the next hop.
- **Example:** If packet is for 192.168.2.5, router checks its table → forwards to correct next router.

👉 **In short:**

- **Delivery** = end-to-end (host to final host).
- **Forwarding** = router-to-router (step in the journey).

2. Unicast Routing Protocol

- **Definition:** A protocol used for sending packets from **one source to one specific destination**.
- **How it works:**

- Each router maintains a **routing table**.
- The table tells the router the **next hop** for a given destination IP.
- The routing decision is based on **shortest path or metrics (hop count, bandwidth, delay, etc.)**.

Examples of Unicast Routing Protocols:

- **Distance Vector Protocols:**
 - *RIP (Routing Information Protocol)* → uses hop count as metric.
 - **Link State Protocols:**
 - *OSPF (Open Shortest Path First)* → uses cost (bandwidth).
 - **Path Vector Protocols:**
 - *BGP (Border Gateway Protocol)* → used for interdomain routing on the Internet.
-

3. Link State Routing Protocol

- **Definition:** A type of routing protocol where each router has complete knowledge of the **topology of the network**.
- **Working Steps:**
 1. Each router discovers its neighbors and measures link costs.
 2. It floods **Link State Advertisements (LSAs)** to all routers.
 3. All routers build the same **link-state database (map of the network)**.
 4. Each router independently runs **Dijkstra's Shortest Path Algorithm (SPF)** to compute best routes.
- **Example: OSPF (Open Shortest Path First)** is the most common link state protocol.

👉 Difference from Distance Vector:

- Distance Vector = routers share only their table with neighbors.
- Link State = routers share full network topology and compute routes themselves.